Lab 4 section 3.3: coding-agent exercises, results log

Team ABCDE. Factual record of runs on 27 Oct 2026. Not a reflection.

This log records what was run and what came out, so the team can write the 3.3.4 reflections in their own words. It holds no opinions on behalf of the team. The questions at the end are prompts for the reflection.

Important deviation from the manual. The manual asks for **Cline with a free model**. These runs used a **Claude Code sub-agent running Claude Haiku** (a small, fast model). The agent started with no memory of the project. The prompts, files and measurements follow the kit in ../README.md. If the marker expects Cline specifically, re-run Exercise 1 attempt 1 in Cline (about 30 minutes) and add a row to the table. The setup below makes that easy.

Setup (both exercises)

- An isolated copy of the app was made outside the repo. In it, `CpfGrantService.java` and `ai-exercise/CpfGrantService.skeleton.java` were deleted and the result committed as a baseline. The project no longer compiled, as intended.
- The agent was told to work only inside that copy, not to read the original elsewhere, and not to use git history. It listed every file it read (below). We cannot fully prove it looked nowhere else, but its file list contains only allowed files, and its code differs structurally from the original (see the diff).
- Build: `mvn -o test` (Spring Boot 3.3, Java 17).

Exercise 1 (3.3.2): regenerate `CpfGrantService` from its tests

Prompt (attempt 1, verbatim)

> The control class `com.abcde.hdbresale.service.CpfGrantService` has been deleted. Recreate it so the project compiles and every test in `CpfGrantServiceBlackBoxTest` and `CpfGrantServiceTest` passes. It implements UC-06 (estimate the CPF Housing Grant, EHG and PHG grants, down payment, loan, monthly instalment and affordability for one flat) and UC-07 (save, read and delete the user's financial profile). Use the rules in `grant-rules.yml` through `GrantRules`; do not hard-code amounts. Read the files I listed before writing code. Do not change any test.

Budget: at most 4 test runs. The agent was told it could edit only `CpfGrantService.java`.

Files given and files read

Given: both test classes, `GrantRules.java`, `grant-rules.yml`, the three DTOs, `CpfController.java`, `PostalSectorDirectory.java`, `docs/lab4/Lab4-Test-Report.md` section 2, and the Part 2 class diagram source (`part2-detail-valuation-cpf.puml`).

Read by the agent, in order: the files above, then on its own initiative `CpfGrant`, `Flat`, `MaritalStatus`, `GeoPoint`, `FinancialProfile`, `ApiException`, `FinancialProfileRepository` and `FlatRepository`. After the first compile error it also read `ProfileDto` and `ProfileService`.

Note: the class diagram gave the agent every public method signature. That is closer to the kit's attempt 3 (skeleton) than a pure "tests only" run.

Attempts

Attempt	Model / tool	Prompt	Test runs	Result (of 35 CPF tests)	Full suite (68)	Time	Files changed
1	Claude Haiku, Claude Code sub-agent	above	2	Run 1: compile error. Run 2: **35 / 35 pass**	**68 / 68 pass**	about 3 min	`CpfGrantService.java` only
2	not needed						
3	not needed						

The run 1 compile error: `ProfileService.java:39` calls `cpfGrantService.savedProfileUpdatedAt(Long)`. That method is not in the prompt or the class diagram. The agent read `ProfileService`, added the method, and the second run passed.

What the tests did not catch

All tests passed, so the code was then checked by hand against the original and probed with four extra tests (UntestedBehaviourProbe.java). These tests are **not** part of the app's suite.

Probe	What it checks	Original class	AI class
P1	A parent distance comes with an "approximate (postal sector)" disclaimer	pass	**fail**: disclaimer dropped
P2	The "first-time buyers who are Singapore citizens" assumption is stated	pass	**fail**: dropped
P3	Living with parents plus an unknown parent postal code does not raise an error (PHG uses "live with", so distance is irrelevant)	pass	**fail**: throws 422
P4	The minimum down payment follows the loan-to-value cap (100 − 75 = 25%), not the default down payment. Checked by setting the default to 30% and asking for 25%	pass (25%)	**fail**: forced to 30%

Other differences found by reading the code (full diff, about 300 changed lines out of 245 original and 286 new):

- **No grant amounts hard-coded.** All grant amounts and ceilings come from `GrantRules`. One threshold is hard-coded: COMFORTABLE at ≤ 25% of income. The original hard-codes it as `msr - 5`. The results are the same while MSR is 30%.

- **Three disclaimers dropped**: the two in P1 and P2, plus "EHG needs 12 months of continuous employment" and "grants are applied to the price, which reduces the loan". It added one disclaimer that was not in the original, saying which date the rules take effect from.
- **Added a NO_PRICE check** (422 when a flat has no price). The original has no such check and would throw a NullPointerException in that case.
- **Profile fallback widened.** If the request profile is incomplete, it falls back to the saved profile. The original does that only if the request profile is missing. `saveProfile` now also rejects an incomplete profile.
- **Flat-size rule inverted in form.** It lists the small types ("1 ROOM" to "4 ROOM") and treats every other type as large. The original lists the large types (5-room, Executive, Multi-generation). The results are the same for the app's data. They differ for an unexpected label, which the AI version treats as large and gives the smaller grant.
- **NFR-6 kept in behaviour.** The parent postal code goes only to `PostalSectorDirectory`, never to an external API. The class Javadoc no longer says so.
- **Less Javadoc.** Public methods have none, apart from the class comment and short helper comments. Lab 4 asks for Javadoc on key methods.
- **Tests not modified.** Checked with `git status`: the only new file is `CpfGrantService.java`.

The agent's own list of guesses (from its report): the 25% threshold, what counts as "up to four room", how the singles EHG works, that grants reduce the loan (inferred from the tests, not the SRS), rounding, the 422 status for PROFILE_REQUIRED, and all message wording.

Exercise 2 (3.3.3), option A: generate the class diagram for UC-01 and UC-06 from code

Prompt (verbatim)

> Read every class under `backend/src/main/java/com/abcde/hdbresale`. Produce a PlantUML class diagram for the boundary, control and entity classes of UC-06 and UC-01, with stereotypes «boundary», «control», «entity», the key attributes and public methods, and associations with multiplicities. Then list every place where the code differs from the attached Lab 3 diagram.

Attached: the Lab 3 class diagram Parts 1 and 2 (PNG). Same model and tool as Exercise 1, in a fresh session, with a copy of `backend/src/main` only.

Result

Item	Outcome
Files read	All 120 Java classes, by its own report. It did not read the YAML, SQL or CSV files.
Rounds	1
Output	uc01-uc06.puml with 14 classes and 16 associations, plus differences.md with 23 differences
Renders?	Not as delivered: PlantUML failed because this machine has no Graphviz. It rendered after adding one line, `!pragma layout smetana` (PNG).
Invented classes or methods	**None.** Each of the 47 method names and 14 class names in the diagram was searched for in the code, and all exist.

Item	Outcome
Spot-checked differences	4 of 23 checked against the code and the Lab 3 image, and all 4 were correct. #13: `submitFinancialProfile` is really a GET that returns 204. #11: `CommuteRoute` is not referenced anywhere (dead code). #5: there is a 16-thread pool. #10: the Lab 3 diagram shows the facade but not the proxy.
Omissions	The diagram leaves out `LivabilityScoringStrategy`, `WeightedSumScoringStrategy`, `ExternalApiGateway` and `CachingApiGatewayProxy`, because they are not boundary, control or entity classes. As a result **neither design pattern (Strategy, Facade + Proxy) appears**, although the Lab 3 diagram shows both.
Level of detail	Entities list every getter and setter (`FinancialProfile` has 13 methods), and the layout is very wide (4096 px). The hand-made Lab 5 diagrams in `docs/design/class-diagrams/` are tighter.
Judgement calls	It gave `SearchSessionStore` and `PostalSectorDirectory` the stereotype «control». The Lab 3 diagram uses «container» for the store. It inferred multiplicities from the code, not from the SRS.

Two of its differences point to real gaps in the Lab 3 diagram that the hand-made Lab 5 diagrams already fix: `PostalSectorDirectory` was missing, and `GrantRules` was missing `cpfHousingGrant`.

How to reproduce

```
        git checkout -b ai-exercise-cpf
        rm backend/src/main/java/com/abcde/hdbresale/service/CpfGrantService.java
        # give the agent the prompt above, then:
        cd backend && mvn test -Dtest='CpfGrantService*'   # 35 tests
        mvn test                                           # 68 tests
        cp ../docs/lab4/ai-exercise/results/ex1/UntestedBehaviourProbe.java
src/test/java/com/abcde/hdbresale/service/
        mvn test -Dtest=UntestedBehaviourProbe             # the 4 probes
        git checkout main
```

Questions to answer in your own words (3.3.4)

Answer from the log above and, if you re-run it, from your own run. About half a page per exercise.

Exercise 1

1. The agent passed every test on its second run. Did that surprise you? Why might it have been so easy here? Think about what the class diagram and the test names gave away.
2. All 68 tests passed, yet four behaviours differ from your design (P1 to P4). What does that say about your test suite? Which of P1 to P4 would you add to the real suite, and which do you think don't matter?
3. Is P4 (minimum down payment from the default instead of the LTV cap) a bug, a different reading of the requirement, or both? Which source would you check?
4. The AI version added a NO_PRICE check that your original lacks. Should you adopt it?

5. Would you merge the AI version? What would you have to change first, for example disclaimers and Javadoc?
6. If it had failed, what would you have tried next: failure output, the skeleton, smaller classes, or more tests?
7. How would Cline with a free model likely compare? If you re-ran it, what actually happened?

Exercise 2

8. Why did you pick option A? Did the output save you work on the Lab 5 diagrams?
9. The generated diagram left out both design patterns because they are not boundary, control or entity classes. Is that a problem with the prompt, with the agent, or a fair reading of what you asked?
10. Which of the 23 differences are real design drift, and which are just detail the Lab 3 diagram left out on purpose (getters, thread pools)?
11. How much would you trust a generated diagram without checking it? How long did checking take compared with drawing it by hand?
12. What would you change in the prompt next time, for example “include strategy and facade classes” or “public methods only, no getters”?